

出流的指针。如果本对象当前关联到一个输出流，则返回的就是指向这个流的指针，如果对象未关联到流，则返回空指针。tie 的第二个版本接受一个指向 ostream 的指针，将自己关联到此 ostream。即，x.tie(&o) 将流 x 关联到输出流 o。

我们既可以将一个 istream 对象关联到另一个 ostream，也可以将一个 ostream 关联到另一个 ostream:

```
cin.tie(&cout);           // 仅仅是用来展示：标准库将 cin 和 cout 关联在一起
// old_tie 指向当前关联到 cin 的流（如果有的话）
ostream *old_tie = cin.tie(nullptr); // cin 不再与其他流关联
// 将 cin 与 cerr 关联；这不是一个好主意，因为 cin 应该关联到 cout
cin.tie(&cerr);           // 读取 cin 会刷新 cerr 而不是 cout
cin.tie(old_tie);         // 重建 cin 和 cout 间的正常关联
```

在这段代码中，为了将一个给定的流关联到一个新的输出流，我们将新流的指针传递给了 tie。为了彻底解开流的关联，我们传递了一个空指针。每个流同时最多关联到一个流，但多个流可以同时关联到同一个 ostream。

8.2 文件输入输出



头文件 fstream 定义了三个类型来支持文件 IO: ifstream 从一个给定文件读取数据，ofstream 向一个给定文件写入数据，以及 fstream 可以读写给定文件。在 17.5.3 节中（第 676 页）我们将介绍如何对同一个文件流既读又写。

这些类型提供的操作与我们之前已经使用过的对象 cin 和 cout 的操作一样。特别是，我们可以用 IO 运算符（<<和>>）来读写文件，可以用 getline（参见 3.2.2 节，第 79 页）从一个 ifstream 读取数据，包括 8.1 节中（第 278 页）介绍的内容也都适用于这些类型。

除了继承自 istream 类型的行为之外，fstream 中定义的类型还增加了一些新的成员来管理与流关联的文件。在表 8.3 中列出了这些操作，我们可以对 fstream、ifstream 和 ofstream 对象调用这些操作，但不能对其他 IO 类型调用这些操作。

表 8.3: fstream 特有的操作	
<i>fstream</i> fstrm;	创建一个未绑定的文件流。 <i>fstream</i> 是头文件 <i>fstream</i> 中定义的一个类型
<i>fstream</i> fstrm(s);	创建一个 <i>fstream</i> ，并打开名为 s 的文件。s 可以是 string 类型，或者是一个指向 C 风格字符串的指针（参见 3.5.4 节，第 109 页）。这些构造函数都是 explicit 的（参见 7.5.4 节，第 265 页）。默认的文件模式 mode 依赖于 <i>fstream</i> 的类型
<i>fstream</i> fstrm(s, mode);	与前一个构造函数类似，但按指定 mode 打开文件
fstrm.open(s)	打开名为 s 的文件，并将文件与 fstrm 绑定。s 可以是一个 string 或一个指向 C 风格字符串的指针。默认的文件 mode 依赖于 <i>fstream</i> 的类型。返回 void
fstrm.close()	关闭与 fstrm 绑定的文件。返回 void
fstrm.is_open()	返回一个 bool 值，指出与 fstrm 关联的文件是否成功打开且尚未关闭

317 8.2.1 使用文件流对象



当我们想要读写一个文件时，可以定义一个文件流对象，并将对象与文件关联起来。每个文件流类都定义了一个名为 `open` 的成员函数，它完成一些系统相关的操作，来定位给定的文件，并视情况打开为读或写模式。

创建文件流对象时，我们可以提供文件名（可选的）。如果提供了一个文件名，则 `open` 会自动被调用：

```
ifstream in(ifile);           // 构造一个 ifstream 并打开给定文件
ofstream out;                 // 输出文件流未关联到任何文件
```

这段代码定义了一个输入流 `in`，它被初始化为从文件读取数据，文件名由 `string` 类型的参数 `ifile` 指定。第二条语句定义了一个输出流 `out`，未与任何文件关联。在新 C++ 标准中，文件名既可以是库类型 `string` 对象，也可以是 C 风格字符数组（参见 3.5.4 节，第 109 页）。旧版本的标准库只允许 C 风格字符数组。

C++
11

用 `fstream` 代替 `iostream&`

我们在 8.1 节（第 279 页）已经提到过，在要求使用基类型对象的地方，我们可以用继承类型的对象来替代。这意味着，接受一个 `iostream` 类型引用（或指针）参数的函数，可以用一个对应的 `fstream`（或 `sstream`）类型来调用。也就是说，如果有一个函数接受一个 `ostream&` 参数，我们在调用这个函数时，可以传递给它一个 `ofstream` 对象，对 `istream&` 和 `ifstream` 也是类似的。

例如，我们可以用 7.1.3 节中的 `read` 和 `print` 函数来读写命名文件。在本例中，我们假定输入和输出文件的名字是通过传递给 `main` 函数的参数来指定的（参见 6.2.5 节，第 196 页）：

```
ifstream input(argv[1]);      // 打开销售记录文件
ofstream output(argv[2]);     // 打开输出文件
Sales_data total;             // 保存销售总额的变量
if (read(input, total)) {     // 读取第一条销售记录
    Sales_data trans;         // 保存下一条销售记录的变量
    while(read(input, trans)) { // 读取剩余记录
        if (total.isbn() == trans.isbn()) // 检查 isbn
            total.combine(trans);         // 更新销售总额
        else {
            print(output, total) << endl; // 打印结果
            total = trans;                // 处理下一本书
        }
    }
    print(output, total) << endl; // 打印最后一本书的销售总额
} else                          // 文件中无输入数据
    cerr << "No data?!" << endl;
```

除了读写的是命名文件外，这段程序与 229 页的加法程序几乎是完全相同的。重要的部分是对 `read` 和 `print` 的调用。虽然两个函数定义时指定的形参分别是 `istream&` 和 `ostream&`，但我们可以向它们传递 `fstream` 对象。

318 成员函数 `open` 和 `close`

如果我们定义了一个空文件流对象，可以随后调用 `open` 来将它与文件关联起来：

```
ifstream in(ifile);           // 构筑一个 ifstream 并打开给定文件
ofstream out;                 // 输出文件流未与任何文件相关联
out.open(ifile + ".copy");    // 打开指定文件
```

如果调用 `open` 失败, `failbit` 会被置位(参见 8.1.2 节, 第 280 页)。因为调用 `open` 可能失败, 进行 `open` 是否成功的检测通常是一个好习惯:

```
if (out)           // 检查 open 是否成功
    // open 成功, 我们可以使用文件了
```

这个条件判断与我们之前将 `cin` 用作条件相似。如果 `open` 失败, 条件会为假, 我们就不会去使用 `out` 了。

一旦一个文件流已经打开, 它就保持与对应文件的关联。实际上, 对一个已经打开的文件流调用 `open` 会失败, 并会导致 `failbit` 被置位。随后的试图使用文件流的操作都会失败。为了将文件流关联到另外一个文件, 必须首先关闭已经关联的文件。一旦文件成功关闭, 我们可以打开新的文件:

```
in.close();           // 关闭文件
in.open(ifile + "2"); // 打开另一个文件
```

如果 `open` 成功, 则 `open` 会设置流的状态, 使得 `good()` 为 `true`。

自动构造和析构

考虑这样一个程序, 它的 `main` 函数接受一个要处理的文件列表(参见 6.2.5 节, 第 196 页)。这种程序可能会有如下的循环:

```
// 对每个传递给程序的文件执行循环操作
for (auto p = argv + 1; p != argv + argc; ++p) {
    ifstream input(*p); // 创建输出流并打开文件
    if (input) {        // 如果文件打开成功, “处理” 此文件
        process(input);
    } else
        cerr << "couldn't open: " + string(*p);
} // 每个循环步 input 都会离开作用域, 因此会被销毁
```

每个循环步构造一个新的名为 `input` 的 `ifstream` 对象, 并打开它来读取给定的文件。像之前一样, 我们检查 `open` 是否成功。如果成功, 将文件传递给一个函数, 该函数负责读取并处理输入数据。如果 `open` 失败, 打印一条错误信息并继续处理下一个文件。

因为 `input` 是 `while` 循环的局部变量, 它在每个循环步中都要创建和销毁一次(参见 5.4.1 节, 第 165 页)。当一个 `fstream` 对象离开其作用域时, 与之关联的文件会自动关闭。在下一步循环中, `input` 会再次被创建。



当一个 `fstream` 对象被销毁时, `close` 会自动被调用。

8.2.1 节练习

319

练习 8.4: 编写函数, 以读模式打开一个文件, 将其内容读入到一个 `string` 的 `vector` 中, 将每一行作为一个独立的元素存于 `vector` 中。

练习 8.5: 重写上面的程序, 将每个单词作为一个独立的元素进行存储。

练习 8.6: 重写 7.1.1 节的书店程序(第 229 页), 从一个文件中读取交易记录。将文件名作为一个参数传递给 `main`(参见 6.2.5 节, 第 196 页)。

8.2.2 文件模式

每个流都有一个关联的文件模式（file mode），用来指出如何使用文件。表 8.4 列出了文件模式和它们的含义。

表 8.4: 文件模式	
in	以读方式打开
out	以写方式打开
app	每次写操作前均定位到文件末尾
ate	打开文件后立即定位到文件末尾
trunc	截断文件
binary	以二进制方式进行 IO

无论用哪种方式打开文件，我们都可以指定文件模式，调用 open 打开文件时可以用一个文件名初始化流来隐式打开文件时也可以。指定文件模式有如下限制：

- 只可以对 ofstream 或 fstream 对象设定 out 模式。
- 只可以对 ifstream 或 fstream 对象设定 in 模式。
- 只有当 out 也被设定时才可设定 trunc 模式。
- 只要 trunc 没被设定，就可以设定 app 模式。在 app 模式下，即使没有显式指定 out 模式，文件也总是以输出方式被打开。
- 默认情况下，即使我们没有指定 trunc，以 out 模式打开的文件也会被截断。为了保留以 out 模式打开的文件的内容，我们必须同时指定 app 模式，这样只会将数据追加写到文件末尾；或者同时指定 in 模式，即打开文件同时进行读写操作（参见 17.5.3 节，第 676 页，将介绍对同一个文件既进行输入又进行输出的方法）。
- ate 和 binary 模式可用于任何类型的文件流对象，且可以与其他任何文件模式组合使用。

每个文件流类型都定义了一个默认的文件模式，当我们未指定文件模式时，就使用此默认模式。与 ifstream 关联的文件默认以 in 模式打开；与 ofstream 关联的文件默认以 out 模式打开；与 fstream 关联的文件默认以 in 和 out 模式打开。

320 以 out 模式打开文件会丢弃已有数据

默认情况下，当我们打开一个 ofstream 时，文件的内容会被丢弃。阻止一个 ofstream 清空给定文件内容的方法是同时指定 app 模式：

```
// 在这几条语句中，file1 都被截断
ofstream out("file1"); // 隐含以输出模式打开文件并截断文件
ofstream out2("file1", ofstream::out); // 隐含地截断文件
ofstream out3("file1", ofstream::out | ofstream::trunc);
// 为了保留文件内容，我们必须显式指定 app 模式
ofstream app("file2", ofstream::app); // 隐含为输出模式
ofstream app2("file2", ofstream::out | ofstream::app);
```



保留被 ofstream 打开的文件中已有数据的唯一方法是显式指定 app 或 in 模式。

每次调用 open 时都会确定文件模式

对于一个给定流，每当打开文件时，都可以改变其文件模式。

```
ofstream out; // 未指定文件打开模式
out.open("scratchpad"); // 模式隐含设置为输出和截断
out.close(); // 关闭 out，以便我们将其用于其他文件
out.open("precious", ofstream::app); // 模式为输出和追加
out.close();
```

第一个 open 调用未显式指定输出模式，文件隐式地以 out 模式打开。通常情况下，out 模式意味着同时使用 trunc 模式。因此，当前目录下名为 scratchpad 的文件的内容将被清空。当打开名为 precious 的文件时，我们指定了 append 模式。文件中已有的数据都得以保留，所有写操作都在文件末尾进行。

在每次打开文件时，都要设置文件模式，可能是显式地设置，也可能是隐式地设置。当程序未指定模式时，就使用默认值。

8.2.2 节练习

练习 8.7: 修改上一节的书店程序，将结果保存到一个文件中。将输出文件名作为第二个参数传递给 main 函数。

练习 8.8: 修改上一题的程序，将结果追加到给定的文件末尾。对同一个输出文件，运行程序至少两次，检验数据是否得以保留。

8.3 string 流

321

sstream 头文件定义了三个类型来支持内存 IO，这些类型可以向 string 写入数据，从 string 读取数据，就像 string 是一个 IO 流一样。

istringstream 从 string 读取数据，ostreamstringstream 向 string 写入数据，而头文件 stringstream 既可从 string 读数据也可向 string 写数据。与 fstream 类型类似，头文件 sstream 中定义的类型都继承自我们已经使用过的 iostream 头文件中定义的类型。除了继承得来的操作，sstream 中定义的类型还增加了一些成员来管理与流相关联的 string。表 8.5 列出了这些操作，可以对 stringstream 对象调用这些操作，但不能对其他 IO 类型调用这些操作。

表 8.5: stringstream 特有的操作	
sstream strm;	strm 是一个未绑定的 stringstream 对象。sstream 是头文件 sstream 中定义的一个类型
sstream strm(s);	strm 是一个 sstream 对象，保存 strings 的一个拷贝。此构造函数是 explicit 的（参见 7.5.4 节，第 265 页）
strm.str()	返回 strm 所保存的 string 的拷贝
strm.str(s)	将 string s 拷贝到 strm 中。返回 void

8.3.1 使用 istringstream

当我们的某些工作是对整行文本进行处理，而其他一些工作是处理行内的单个单词